Data Mining & Machine Learning Project Report: Linear Regression, Decision Trees and kNN

MSc in Software Design with AI | Name: Sampath Reddy Kamsani | Student ID: A00348851

Dataset documentation: (1) Auto MPG: https://archive.ics.uci.edu/dataset/9/auto+mpg

(2) Heart Disease (Cleveland): https://archive.ics.uci.edu/dataset/45/heart+disease. CSV copies are in the data folder.

1. Linear Regression: Car Fuel Efficiency (Auto MPG)

1.1 Objectives

A car importer or fleet buyer wants a rough idea of a car's fuel economy from its specification, before paying for road tests. I also test a polynomial term.

Target variable: mpg (miles per gallon, continuous). Predictors: cylinders, displacement, horsepower, weight, acceleration, model year and origin. I count it as a success if test RMSE is about 3 mpg or lower and R^2 is above 0.8.

1.2 Data Exploration

398 cars, 9 columns. Only horsepower has missing values (6 rows), so I used the median. I dropped name (just a label) and one-hot encoded origin.

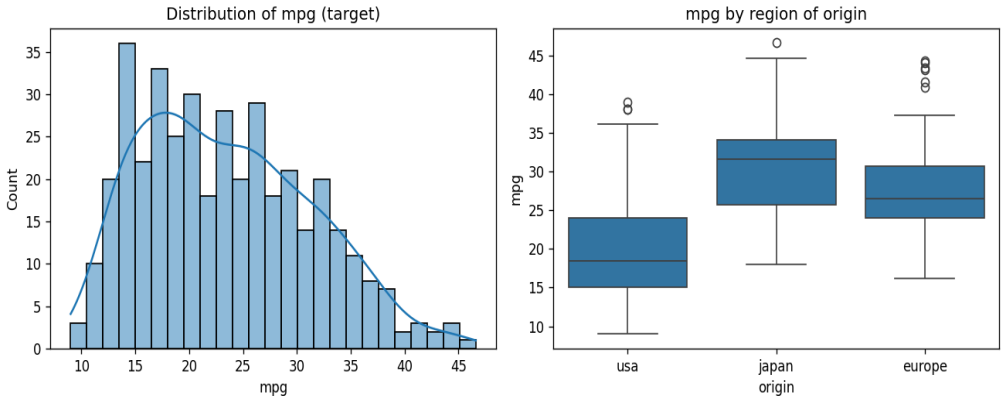


Figure 1: distribution of mpg and mpg by origin

Comment: *mpg is slightly right-skewed (skew 0.46), mostly between 15 and 30, so no transformation was needed. US cars average about 20 mpg, European about 28 and Japanese about 30, so origin looks like a useful predictor.*

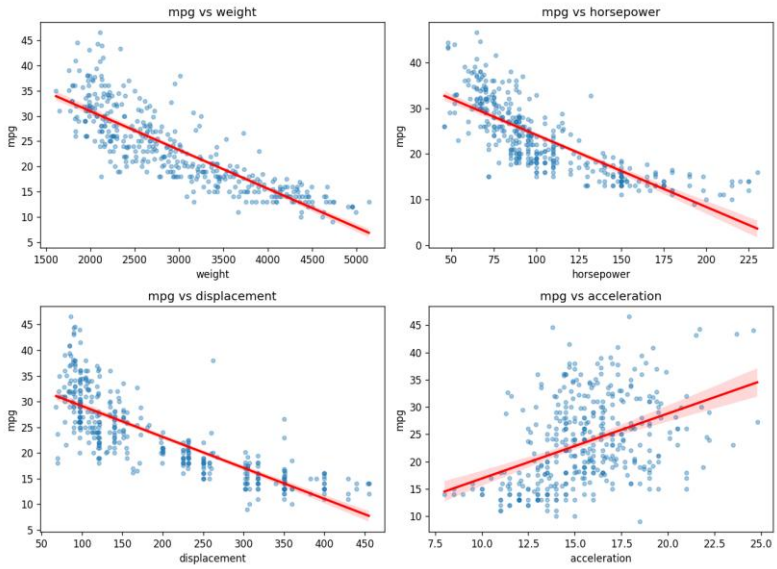


Figure 2: mpg vs key features

Comment: *In Figure 2 weight and horsepower both have a strong negative link with mpg, but the weight plot is curved (the points are above the red line at both ends), so I want to try a squared term. In Figure 3, weight, displacement, cylinders and horsepower correlate with mpg at -0.77 to -0.83 and with each other at 0.84 to 0.95, so*

multicollinearity is likely. Single coefficients may be unreliable, but predictions should be fine. model_year (+0.58) shows efficiency improving over time.

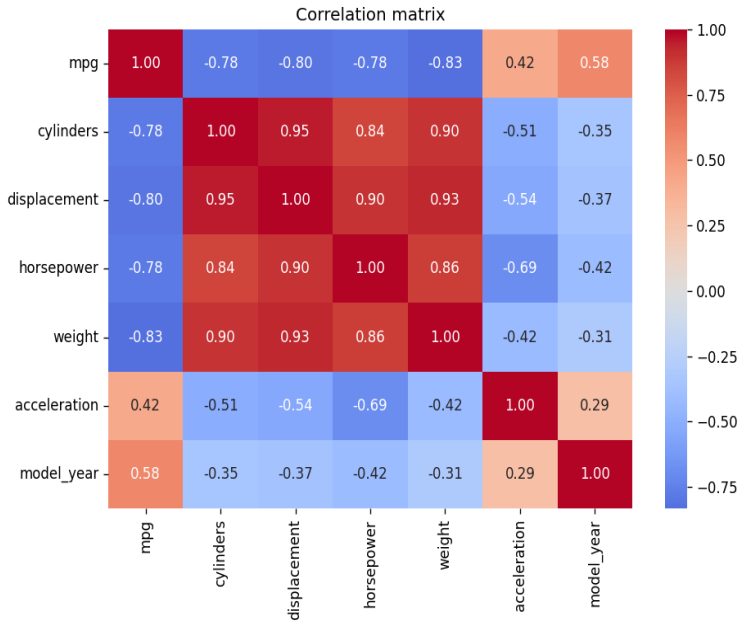


Figure 3: correlation matrix

1.3 Modeling

```
X = df.drop(columns='mpg'); y = df['mpg']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
m2 = LinearRegression().fit(X_train, y_train)                    # Model 2: all features
cross_val_score(LinearRegression(), X_train, y_train, cv=5, scoring='r2')
X_train['weight_sq'] = (X_train['weight']/1000)**2               # polynomial term (Models 3 and 4)
```

Model 2 (all features) has intercept -18.91 and coefficients: model_year +0.826, origin_usa -2.95, origin_japan -0.28 (reference group is Europe), weight -0.0070, horsepower -0.0146, acceleration +0.068, cylinders -0.17 and displacement +0.020. So each newer model year adds about 0.8 mpg, 100 extra lb costs about 0.7 mpg, and US cars are about 2.9 mpg lower. The odd cylinders and displacement values are probably due to collinearity.

Polynomial regression: I think a quadratic is appropriate here. Model 3 (weight + weight², weight in 1000 lb) gives mpg = 62.39 - 18.41·w + 1.67·w², and Model 4 adds weight² (coefficient +2.28) to all the features. Figure 4 shows that for weight alone CV R² is highest at degree 2 (0.691 against 0.674 for a straight line) and does not improve for higher degrees, so I stopped at degree 2.

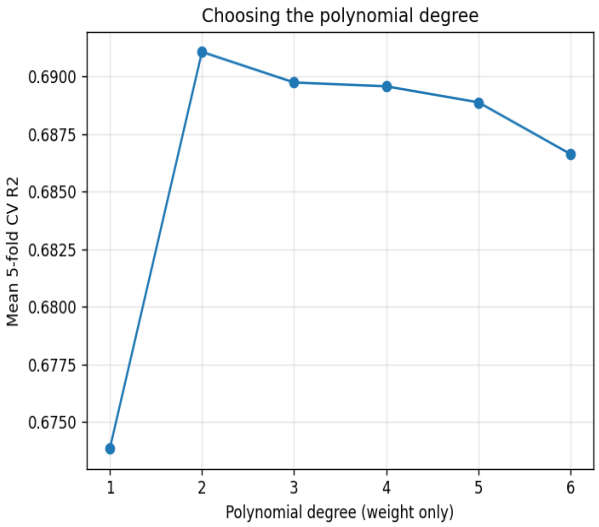


Figure 4: CV R² against polynomial degree

1.4 Evaluation & Conclusion

Model	CV R^2 (5-fold)	CV RMSE	Test R^2	Test RMSE
1: weight only	0.674	4.48	0.723	3.86
2: all features	0.805	3.46	0.845	2.89
3: weight + weight²	0.691	4.34	0.755	3.63
4: all features + weight²	**0.840**	**3.12**	**0.882**	**2.52**

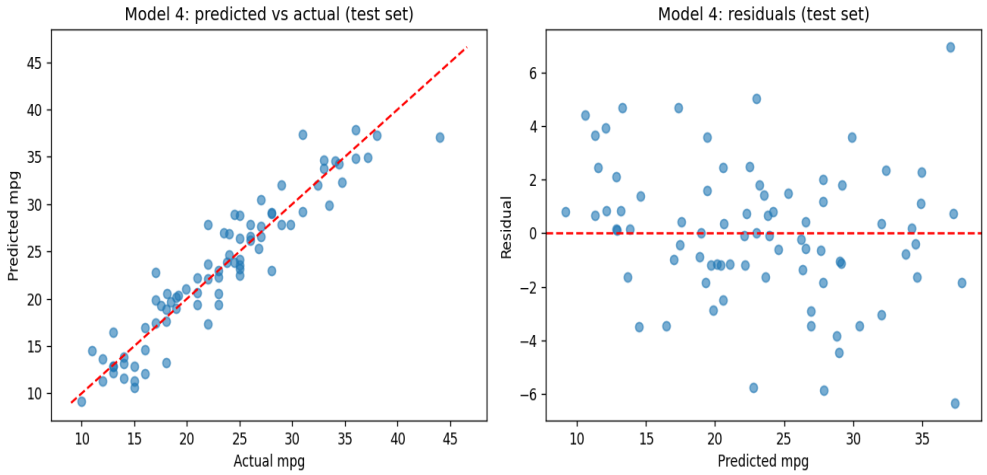


Figure 5: Model 4 predicted vs actual and residuals (test set)

Comment: *the predictions follow the diagonal closely and the residuals show no clear pattern, though they spread out a little more for low-mpg (heavy) cars.*

The squared term helped both models: simple test R^2 0.72 to 0.76, full model CV R^2 0.805 to 0.840 (test 0.845 to 0.882). Model 4 meets my objective: about 2.5 mpg RMSE on unseen cars, with CV and test results close, so overfitting looks unlikely. The data is only from 1970 to 1982 and the test set has just 80 cars, so the numbers are a bit noisy.

2. Decision Trees: Heart Disease Diagnosis

2.1 Objectives

A cardiology clinic wants to decide which patients should get expensive follow-up tests first, using routine measurements. Missing a sick patient is worse than an extra check, so recall matters as much as accuracy, and I would like a model a doctor can read.

Target variable: disease (1 = heart disease present, 0 = absent), predicted from 13 clinical features (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal). I count it as a success if test accuracy is well above the 54% majority baseline with reasonable recall.

2.2 Data Exploration

There are 303 patients with no missing values and one duplicate row, which I removed (302 left: 164 healthy, 138 with disease). Important: in this CSV, target = 1 is actually the healthy group (higher max heart rate 158 vs 139, lower ST depression 0.58 vs 1.59), so I created disease = 1 - target. The classes are fairly balanced, so accuracy is fair.

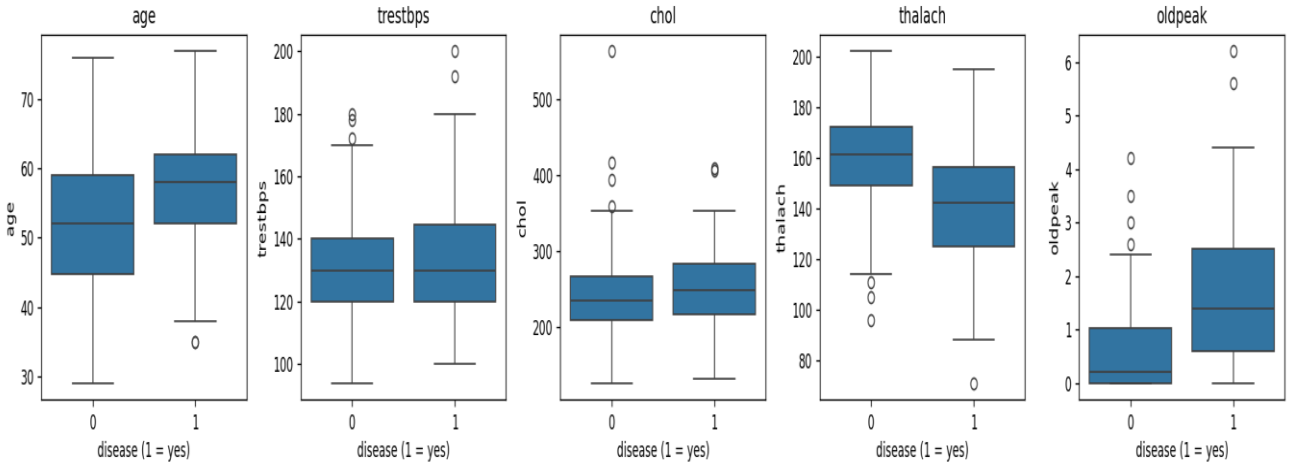


Figure 6: numeric predictors by disease status

Comment: *thalach is lower for patients with disease (median 142 vs 161), oldpeak is higher (1.4 vs 0.2) and age is a bit higher (58 vs 52). chol and trestbps overlap a lot, so they look weak alone.*

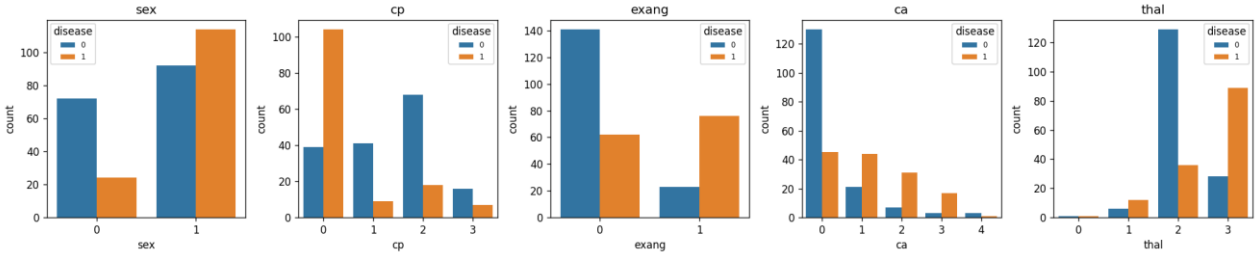


Figure 7: categorical predictors by disease status

Comment: *disease is more common in men (55% vs 25%), with exercise angina (77% vs 31%) and with more major vessels coloured (ca: 26% at 0, over 80% at 2 to 3). thal code 3 has 76% disease against 22% for code 2, and chest-pain code 0 has the highest rate (73%).*

2.3 Modeling

```
X = df.drop(columns='disease'); y = df['disease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    {'max_depth': range(1, 11), 'min_samples_leaf': [1, 3, 5, 10]}, cv=5)
grid.fit(X_train, y_train);  tree = grid.best_estimator_
```

A tree with no limits reached depth 11 with 44 leaves. It got 100% on the training data but only 75.1% in cross-validation, so it overfits. Grid search with 5-fold CV chose max_depth = 4 and min_samples_leaf = 5 (CV accuracy 0.813): 13 leaves.

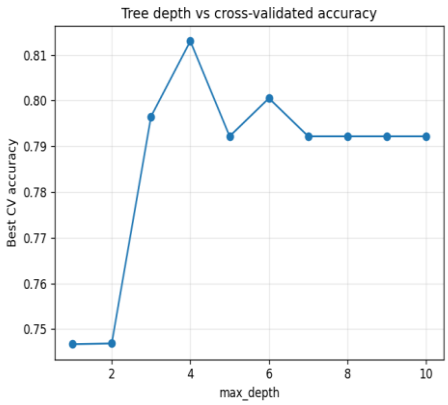


Figure 8: depth vs CV accuracy

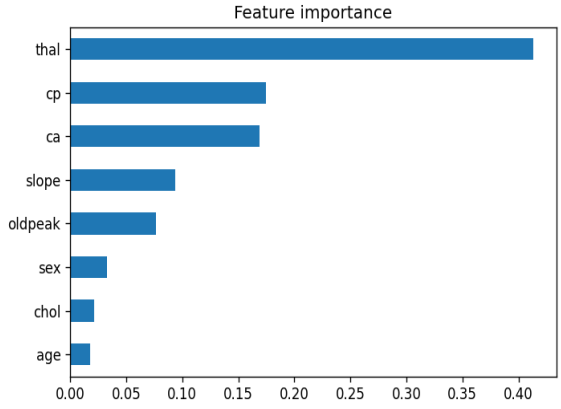


Figure 9: feature importance

Comment: *CV accuracy rises sharply up to depth 4 and then drops and stays near 0.79, because deeper trees fit noise. The top features are thal (0.41), cp (0.17) and ca (0.17), matching the exploration.*

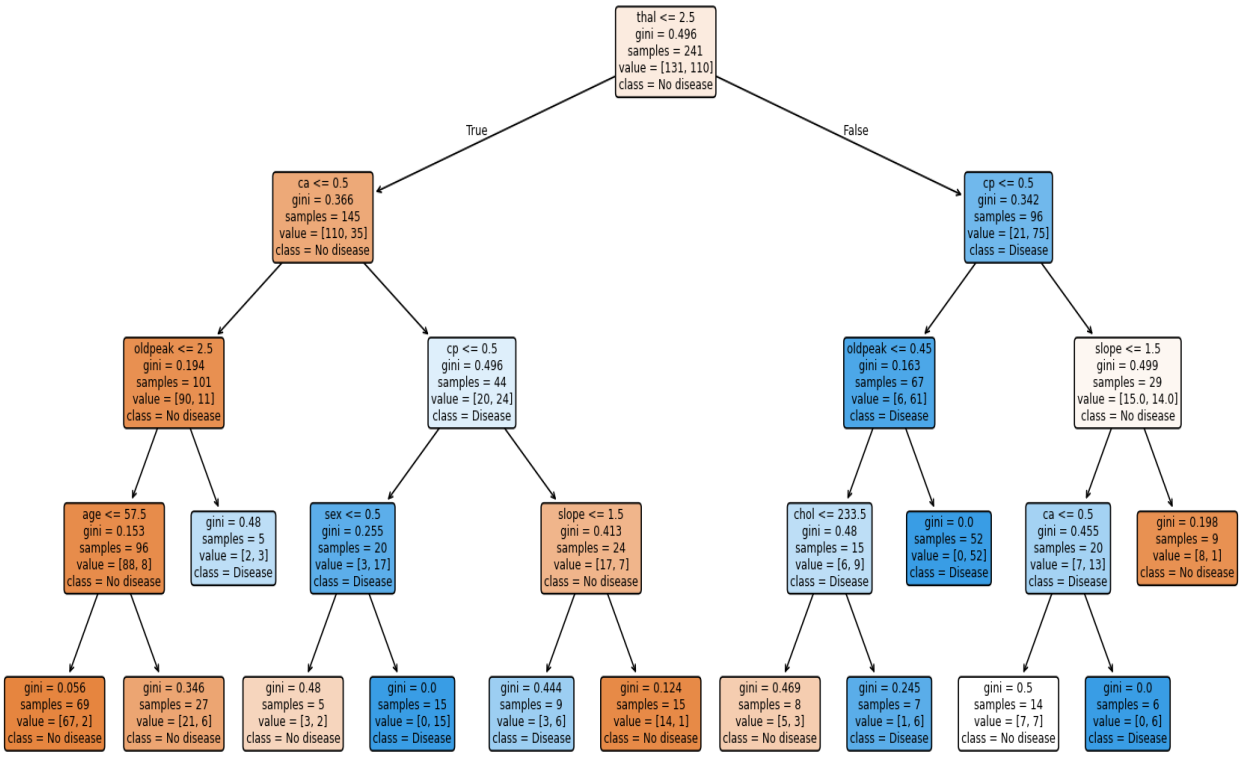


Figure 10: the final decision tree (depth 4)

2.4 Evaluation & Conclusion

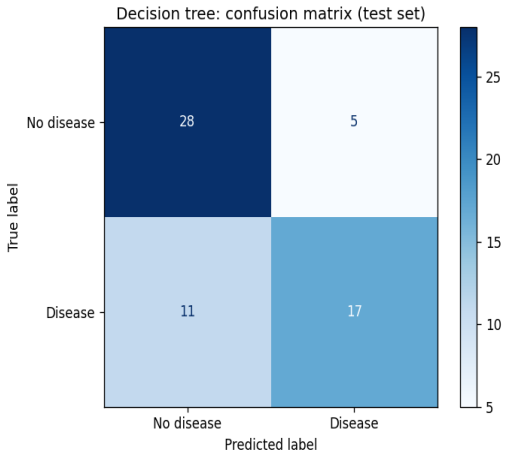


Figure 11: confusion matrix (test, n = 61)

Test accuracy is 0.738 (45 of 61) against a baseline of 0.541. For the disease class precision is 0.77 and recall is 0.61: 11 of the 28 sick patients were missed (false negatives) and 5 healthy patients were flagged. The tree beats the baseline by about 20 points with a few readable rules, and the main drivers (thal, cp, ca) make medical sense. But 61% recall is too low for screening on its own. Test accuracy is also below the CV accuracy (0.81), probably because tuning on CV scores is optimistic and one patient is worth 1.6 points in a test set of 61.

3. kNN: Heart Disease Diagnosis (same dataset)

3.3 Modeling

```
prep = ColumnTransformer([('num', StandardScaler(), ['age','trestbps','chol','thalach','oldpeak','ca']),
                          ('cat', OneHotEncoder(handle_unknown='ignore'), ['cp','restecg','slope','thal'])],
                         remainder='passthrough')
knn_pipe = lambda k: Pipeline([('prep', prep), ('knn', KNeighborsClassifier(n_neighbors=k))])
cv_acc = [cross_val_score(knn_pipe(k), X_train, y_train, cv=5).mean() for k in range(1, 31)]
```

Same cleaning and same stratified split (random_state = 42), so both models face the same 61 test patients. Scaling was essential: 5-fold CV accuracy at k = 5 is 0.639 on raw features but 0.842 after standardising numeric columns and one-hot encoding nominal ones, since chol and thalach otherwise dominate the distance.

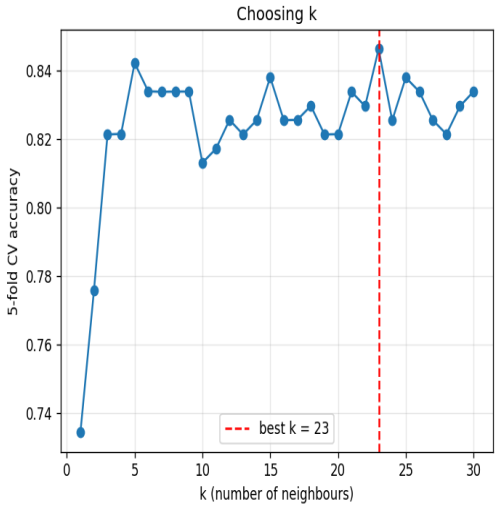


Figure 12: k vs CV accuracy

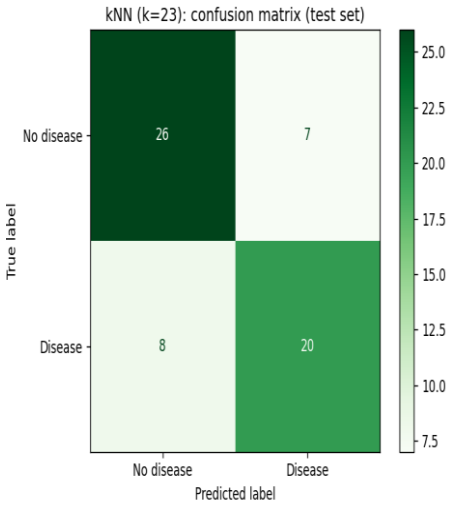


Figure 13: KNN confusion matrix (test)

Comment: *k = 1 overfits (about 0.73). Accuracy rises until k = 5 and then stays between about 0.81 and 0.85, with the best at k = 23 (0.846), only slightly above k = 5 (0.842). I kept k = 23 because it gives a smoother boundary. The confusion matrix shows 20 of 28 sick patients found (8 missed) and 7 healthy patients flagged.*

3.4 Evaluation & Comparison with Decision Tree

Metric	Decision tree	kNN (k = 23)
10-fold CV accuracy (train)	0.776	0.817
Test accuracy	0.738	0.754
Test precision (disease)	0.773	0.741
Test recall (disease)	0.607	0.714
False negatives (missed cases)	11	8

kNN is better in cross-validation (+4 points) and slightly better on the test set (+1.6 points, which is a single patient, so not conclusive by itself). What matters more to me is recall: kNN misses 8 sick patients and the tree misses 11. The tree has slightly higher precision and is much easier to explain, while kNN gives no explanation. Neither gets high enough recall (best 71%) to be used without a doctor. I would pick kNN if catching cases is the priority and the tree if explaining decisions matters more. To improve, I would try a random forest, more data, or a lower decision threshold.